

LM-IDNet: Atomic Implementation Roadmap

From Research Scaffold to Reproducible Adaptive IoT Anomaly Detector

Master's Thesis Engineering Plan

July 21, 2026

Abstract

This document converts Chapter 5 of the LM-IDNet proposal into an executable engineering plan. Every task is intentionally small, has one primary outcome, and is paired with an objective acceptance gate. The plan preserves the proposal's phases while adding software-quality, security, data-governance, numerical-reliability, and research-reproducibility controls needed for a defensible thesis artifact. A task may be marked Done only when its test passes and the named evidence is retained in version control or an immutable experiment directory.

Contents

1	Execution and Control Model	7
1.1	Definitions of Done	7
1.2	Traceability	7
1.3	Mandatory Stage Gates	7
2	F0: Foundational Software Engineering and Research Controls	7
2.0.1	F0.01: Record the baseline state	7
2.0.2	F0.02: Adopt a source-layout package	8
2.0.3	F0.03: Lock runtime and development dependencies	8
2.0.4	F0.04: Centralize typed configuration	8
2.0.5	F0.05: Separate configuration from secrets	8
2.0.6	F0.06: Standardize the command-line interface	8
2.0.7	F0.07: Configure formatting and linting	9
2.0.8	F0.08: Introduce static type checking	9
2.0.9	F0.09: Define the testing architecture	9
2.0.10	F0.10: Set coverage policy	9
2.0.11	F0.11: Add property-based tests	9
2.0.12	F0.12: Create continuous integration	9
2.0.13	F0.13: Configure pre-commit checks	10
2.0.14	F0.14: Harden container builds	10
2.0.15	F0.15: Make container execution reproducible	10
2.0.16	F0.16: Adopt structured logging	10
2.0.17	F0.17: Define exception and failure semantics	10

2.0.18	F0.18: Version schemas and artifacts	11
2.0.19	F0.19: Create immutable experiment manifests	11
2.0.20	F0.20: Establish research-data governance	11
2.0.21	F0.21: Protect raw evidence	11
2.0.22	F0.22: Define the security threat model	11
2.0.23	F0.23: Write architecture decision records	12
2.0.24	F0.24: Define reproducibility commands	12
2.0.25	F0.25: Create contribution and review policy	12
3	P0: Research Setup and Reproducibility	12
3.0.1	P0.01: Inventory source captures	12
3.0.2	P0.02: Verify capture integrity	12
3.0.3	P0.03: Confirm dataset provenance and benign assumptions	13
3.0.4	P0.04: Resolve the October 10–11 split decision	13
3.0.5	P0.05: Freeze temporal partitions	13
3.0.6	P0.06: Create dataset fingerprints	13
3.0.7	P0.07: Centralize deterministic seeds	13
3.0.8	P0.08: Create run-directory isolation	13
3.0.9	P0.09: Separate preprocessing from downstream commands	14
3.0.10	P0.10: Build a tiny public smoke fixture	14
3.0.11	P0.11: Generate the initial dataset report	14
3.0.12	P0.12: Create an end-to-end smoke command	14
4	P1: Data Processing Hardening	14
4.0.1	P1.01: Define canonical category semantics	14
4.0.2	P1.02: Normalize category input	15
4.0.3	P1.03: Harden timestamp parsing	15
4.0.4	P1.04: Specify window boundaries	15
4.0.5	P1.05: Preserve window timestamps	15
4.0.6	P1.06: Represent silent windows	15
4.0.7	P1.07: Represent missing coverage	15
4.0.8	P1.08: Validate count values	16
4.0.9	P1.09: Validate temporal streams	16
4.0.10	P1.10: Make PCAP parsing resource-safe	16
4.0.11	P1.11: Make export atomic	16
4.0.12	P1.12: Export canonical JSON	16
4.0.13	P1.13: Export modeling matrices	17
4.0.14	P1.14: Load matrices by partition	17
4.0.15	P1.15: Compute descriptive statistics	17
4.0.16	P1.16: Compute dispersion diagnostics	17

4.0.17	P1.17: Report cross-category dependence	17
4.0.18	P1.18: Generate data-quality reports	17
4.0.19	P1.19: Add processing regression fixtures	18
4.0.20	P1.20: Benchmark preprocessing	18
5	P2: Dirichlet–Multinomial Modeling	18
5.0.1	P2.01: Define estimator input contract	18
5.0.2	P2.02: Verify the likelihood formula	18
5.0.3	P2.03: Implement robust initialization	19
5.0.4	P2.04: Harden the fixed-point update	19
5.0.5	P2.05: Define convergence criteria	19
5.0.6	P2.06: Record optimization history	19
5.0.7	P2.07: Activate the modeling stage	19
5.0.8	P2.08: Define the model artifact	19
5.0.9	P2.09: Recover synthetic parameters	20
5.0.10	P2.10: Test permutation equivariance	20
5.0.11	P2.11: Test row-order invariance	20
5.0.12	P2.12: Assess likelihood stability	20
5.0.13	P2.13: Create model-fit diagnostics	20
5.0.14	P2.14: Benchmark estimator scaling	20
6	P3: Likelihood Backend Abstraction	21
6.0.1	P3.01: Define the backend protocol	21
6.0.2	P3.02: Implement the standard gammaln backend	21
6.0.3	P3.03: Build a high-precision oracle	21
6.0.4	P3.04: Specify the LM numerical contract	21
6.0.5	P3.05: Implement paired log-gamma differences	21
6.0.6	P3.06: Assemble the Python LM backend	22
6.0.7	P3.07: Add optional Yu–Shaw comparison	22
6.0.8	P3.08: Test extreme numerical regimes	22
6.0.9	P3.09: Test additive batch consistency	22
6.0.10	P3.10: Implement backend selection	22
6.0.11	P3.11: Create backend benchmarks	23
6.0.12	P3.12: Prototype the optimized LM binding	23
7	P4: Threshold Calibration and Anomaly Scoring	23
7.0.1	P4.01: Enforce fit/calibration isolation	23
7.0.2	P4.02: Compute raw window scores	23
7.0.3	P4.03: Define normalized scores	23
7.0.4	P4.04: Analyze volume confounding	24

7.0.5	P4.05: Calibrate empirical quantiles	24
7.0.6	P4.06: Quantify threshold uncertainty	24
7.0.7	P4.07: Persist threshold artifacts	24
7.0.8	P4.08: Implement online scoring	24
7.0.9	P4.09: Implement anomaly decisions	24
7.0.10	P4.10: Calibrate severity	25
7.0.11	P4.11: Compute category residual explanations	25
7.0.12	P4.12: Validate explanation faithfulness	25
7.0.13	P4.13: Define anomaly event schemas	25
7.0.14	P4.14: Make event output durable	25
7.0.15	P4.15: Generate calibration reports	26
8	P5: Experimental Anomaly Evaluation	26
8.0.1	P5.01: Audit labels and attack data	26
8.0.2	P5.02: Pre-register the evaluation protocol	26
8.0.3	P5.03: Define synthetic injection semantics	26
8.0.4	P5.04: Implement TCP flood injections	26
8.0.5	P5.05: Implement UDP burst injections	27
8.0.6	P5.06: Implement SSDP discovery injections	27
8.0.7	P5.07: Implement ARP scan injections	27
8.0.8	P5.08: Implement cloud-traffic suppression	27
8.0.9	P5.09: Implement mixed composition drift	27
8.0.10	P5.10: Validate anomaly generators	27
8.0.11	P5.11: Implement independent Poisson baseline	28
8.0.12	P5.12: Implement Multinomial baseline	28
8.0.13	P5.13: Implement Gaussian baseline	28
8.0.14	P5.14: Implement Isolation Forest baseline	28
8.0.15	P5.15: Implement One-Class SVM baseline	28
8.0.16	P5.16: Standardize baseline interfaces	28
8.0.17	P5.17: Implement window-level metrics	29
8.0.18	P5.18: Implement event-level metrics	29
8.0.19	P5.19: Estimate uncertainty	29
8.0.20	P5.20: Run sensitivity analysis	29
8.0.21	P5.21: Run ablation studies	29
8.0.22	P5.22: Perform final locked evaluation	29
8.0.23	P5.23: Analyze false positives and negatives	30
8.0.24	P5.24: Generate publication tables and figures	30
9	P6: Adaptive Retraining	30
9.0.1	P6.01: Define adaptation states	30

9.0.2	P6.02: Implement the safe-window predicate	30
9.0.3	P6.03: Implement a bounded safe buffer	30
9.0.4	P6.04: Trigger candidate training	31
9.0.5	P6.05: Train candidates in isolation	31
9.0.6	P6.06: Limit parameter movement	31
9.0.7	P6.07: Recalibrate candidate thresholds safely	31
9.0.8	P6.08: Evaluate candidates in shadow mode	31
9.0.9	P6.09: Define promotion policy	31
9.0.10	P6.10: Promote atomically	32
9.0.11	P6.11: Implement automatic rollback	32
9.0.12	P6.12: Retain model lineage	32
9.0.13	P6.13: Simulate gradual benign drift	32
9.0.14	P6.14: Simulate abrupt benign change	32
9.0.15	P6.15: Test poisoning resistance	32
9.0.16	P6.16: Test evasion near threshold	33
9.0.17	P6.17: Compare adaptation policies	33
9.0.18	P6.18: Generate the adaptation audit report	33
10	P7: Forecasting and Response Support	33
10.0.1	P7.01: Remove synthetic forecasting actuals	33
10.0.2	P7.02: Define composition forecasts	33
10.0.3	P7.03: Implement moving-average total forecasts	34
10.0.4	P7.04: Implement seasonal total forecasts	34
10.0.5	P7.05: Combine total and composition	34
10.0.6	P7.06: Correct the Brier metric contract	34
10.0.7	P7.07: Harden Jensen–Shannon divergence	34
10.0.8	P7.08: Add total-count forecast metrics	34
10.0.9	P7.09: Use rolling-origin evaluation	35
10.0.10	P7.10: Report forecast uncertainty	35
10.0.11	P7.11: Define advisory response taxonomy	35
10.0.12	P7.12: Implement deterministic response rules	35
10.0.13	P7.13: Enforce advisory-only operation	35
10.0.14	P7.14: Audit response provenance	35
10.0.15	P7.15: Generate forecasting and response reports	36
11	P8: Edge Deployment Study	36
11.0.1	P8.01: Define target edge profiles	36
11.0.2	P8.02: Create a benchmark harness	36
11.0.3	P8.03: Benchmark standard Python backend	36
11.0.4	P8.04: Benchmark Python LM backend	36

11.0.5 P8.05: Benchmark optimized LM backend	37
11.0.6 P8.06: Measure bounded-memory streaming	37
11.0.7 P8.07: Measure deadline compliance	37
11.0.8 P8.08: Apply least privilege	37
11.0.9 P8.09: Secure the software supply chain	37
11.0.10 P8.10: Verify offline operation	37
11.0.11 P8.11: Test artifact integrity at deployment	38
11.0.12 P8.12: Test graceful restart and replay	38
11.0.13 P8.13: Test resource exhaustion behavior	38
11.0.14 P8.14: Document deployment	38
11.0.15 P8.15: Generate the edge-readiness report	38
12 Cross-Phase Research and Release Controls	39
12.1 Experiment Validity Checklist	39
12.2 Numerical Reliability Checklist	39
12.3 Release Checklist	39
13 Optional Extensions After the Core Gates	39
14 Final Deliverable Matrix	39
15 Progress Summary	40

1 Execution and Control Model

1.1 Definitions of Done

For every task, “Done” means: (i) implementation and documentation are committed; (ii) the stated automated test or manual protocol passes; (iii) no pre-existing test regresses; (iv) generated evidence is stored under `reports/`, `artifacts/`, or the CI system; and (v) assumptions and deviations are recorded in an architecture decision record (ADR) or experiment manifest. A phase closes only after its exit gate passes.

1.2 Traceability

Use identifiers P0–P8 for Chapter 5 phases and F0 for foundational engineering. Each Git issue, branch, commit, test, experiment manifest, and thesis result should cite at least one task identifier. Maintain `docs/traceability.csv` with columns `requirement`, `task`, `code`, `test`, `artifact`, and `thesis_section`.

1.3 Mandatory Stage Gates

Gate	Required evidence
F0	Clean CI run; locked environment; buildable container; documented architecture, security model, and data policy.
P0	Immutable dataset inventory and temporal split; one-command deterministic dry run; dataset summary.
P1	Validated, time-indexed matrices; silent/missing-window policy; diagnostics report; processing test suite.
P2	Persisted valid model with convergence history; synthetic recovery and persistence tests.
P3	Pluggable backends; high-precision numerical oracle; accuracy and runtime report.
P4	Leakage-free calibration; structured scores, explanations, and anomaly events; calibration report.
P5	Frozen evaluation protocol; baseline comparison with uncertainty; error analysis.
P6	Shadow-tested adaptive model; poisoning controls; promotion and rollback evidence.
P7	Real-data forecasts and policy-controlled advisory responses; evaluation report.
P8	Reproducible edge benchmark; hardened container; deployment and limitations guide.

2 F0: Foundational Software Engineering and Research Controls

2.0.1 F0.01: Record the baseline state

Implementation. Create `docs/baseline.md` recording the current commit, supported commands, passing and failing tests, known defects, and current gaps: inactive modeling, synthetic forecasting, absent scoring, absent LM backend, and CLI inconsistency.

Verification and acceptance gate. Run the existing test suite from a clean checkout and paste the exact command, environment, pass/fail counts, and failure summaries into the document. A second developer must be able to reproduce the same failures.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.2 F0.02: Adopt a source-layout package

Implementation. Define one import strategy for `src/`; add package metadata in `pyproject.toml`; expose a console entry point such as `lm-idnet`; remove runtime `sys.path` mutation after migration.

Verification and acceptance gate. Install the project into a fresh virtual environment in editable mode, execute `python -c "import ..."`, invoke `lm-idnet -help`, and run tests without setting `PYTHONPATH`.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.3 F0.03: Lock runtime and development dependencies

Implementation. Separate runtime, test, documentation, and development dependencies; pin direct and transitive versions using a reproducible lock mechanism; declare the supported Python version.

Verification and acceptance gate. Create two fresh environments from the lock file and compare installed-package manifests byte-for-byte. Run the full tests in both environments.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.4 F0.04: Centralize typed configuration

Implementation. Define a Pydantic configuration model for paths, dates, categories, seeds, estimator settings, calibration, adaptation, and output locations. Reject unknown fields and invalid cross-field combinations.

Verification and acceptance gate. Unit-test missing fields, extra fields, invalid dates, duplicate categories, non-positive window sizes, invalid quantiles, and a known-valid configuration. Snapshot the normalized configuration.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.5 F0.05: Separate configuration from secrets

Implementation. Keep research parameters in versioned files and secrets or host-specific paths in environment variables or ignored local overrides. Add an example environment file containing no credentials.

Verification and acceptance gate. Run a secret scanner over the repository and verify that a clean clone operates with the documented non-secret defaults. Confirm that local overrides are ignored by Git.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.6 F0.06: Standardize the command-line interface

Implementation. Define stable subcommands: `preprocess`, `diagnose`, `train`, `calibrate`, `score`, `evaluate`, `adapt`, `forecast`, and `benchmark`. Make errors non-zero and machine-readable where appropriate.

Verification and acceptance gate. For every subcommand, test `-help`, missing arguments, invalid configuration, and a minimal success case. Assert exit codes and essential output fields.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.7 F0.07: Configure formatting and linting

Implementation. Adopt an automatic formatter and a fast linter with rules for imports, complexity, unsafe constructs, and unused code; store all settings in `pyproject.toml`.

Verification and acceptance gate. Run both tools in check-only mode over source and tests with zero violations. Add a deliberately malformed temporary fixture in a tool test to prove the check fails.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.8 F0.08: Introduce static type checking

Implementation. Add annotations to public interfaces and configure a type checker with strict checks for the core `algorithms`, `models`, and `artifact-schema` modules. Expand strict coverage incrementally to processing and evaluation.

Verification and acceptance gate. Type-check the strict modules with zero errors. Include type-checking in CI and demonstrate that an incompatible count-array argument is detected in a test fixture.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.9 F0.09: Define the testing architecture

Implementation. Configure pytest markers for `unit`, `integration`, `numerical`, `slow`, `pcap`, and `benchmark`. Establish fixture factories for count matrices, windows, configurations, and temporary artifact stores.

Verification and acceptance gate. Run each marker independently; ensure no test is silently uncollected and that the default fast suite excludes only explicitly marked slow or data-heavy tests.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.10 F0.10: Set coverage policy

Implementation. Collect branch coverage for `src/`; set an initial non-regression threshold and higher per-module thresholds for schemas, likelihood code, calibration, and decision logic.

Verification and acceptance gate. CI must fail when coverage drops below the configured threshold. Generate HTML and XML reports and confirm important exception branches are exercised.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.11 F0.11: Add property-based tests

Implementation. Use generated inputs for non-negative count matrices, category vectors, timestamps, and configuration boundaries. Define invariants such as finite outputs, non-negative persisted counts, and category-order preservation.

Verification and acceptance gate. Execute at least 100 generated examples per core invariant with reproducible failure seeds. Store any minimized regression example as a permanent unit test.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.12 F0.12: Create continuous integration

Implementation. Add GitHub Actions jobs for formatting, linting, types, unit tests, numerical tests, coverage, package build, and container build on supported Python versions. Cache only immutable

dependency data.

Verification and acceptance gate. Open a test pull request: verify all jobs pass; then introduce a controlled lint or test failure and verify merging is blocked. Archive the successful workflow URL in docs/baseline.md.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.13 F0.13: Configure pre-commit checks

Implementation. Run formatting, linting, trailing-whitespace, large-file, merge-conflict, YAML/JSON, and secret checks before commits. Exclude only documented generated or research-data paths.

Verification and acceptance gate. Run hooks against all files with zero failures. Attempt to commit a synthetic secret and an oversized dummy PCAP and verify both are rejected.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.14 F0.14: Harden container builds

Implementation. Create a minimal non-root runtime image, pin the base image by digest, use a multi-stage build, exclude raw data and development files, and add a meaningful application health check.

Verification and acceptance gate. Build without network-dependent runtime steps, inspect the image user, run a read-only-root-filesystem smoke test with explicit writable mounts, and scan for critical/high vulnerabilities.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.15 F0.15: Make container execution reproducible

Implementation. Provide a Compose profile for development and a separate benchmark profile with fixed resource limits, read-only configuration mounts, and explicit artifact volumes.

Verification and acceptance gate. On a clean machine or CI runner, build and run the smoke dataset twice; compare normalized artifacts and confirm the container cannot modify configuration or raw-data mounts.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.16 F0.16: Adopt structured logging

Implementation. Replace global logging configuration with a library-safe logger and structured records containing run ID, stage, dataset ID, device ID, model version, and severity; never log packet payloads.

Verification and acceptance gate. Capture logs in tests and assert required fields, stable severity levels, and absence of payload bytes, secrets, and absolute private source paths.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.17 F0.17: Define exception and failure semantics

Implementation. Create domain exceptions for configuration, ingestion, validation, numerical precision, convergence, artifact compatibility, and policy rejection. Ensure commands fail closed when model integrity is uncertain.

Verification and acceptance gate. Unit-test every exception mapping to CLI exit code and message. Corrupt a model checksum and verify scoring stops rather than continuing with a warning.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.18 F0.18: Version schemas and artifacts

Implementation. Assign semantic schema versions to processed datasets, models, thresholds, events, and experiment manifests. Define compatible migrations and reject unknown major versions.

Verification and acceptance gate. Round-trip each schema through JSON; load the previous supported version through a migration test; reject an unsupported future major version.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.19 F0.19: Create immutable experiment manifests

Implementation. For every run, store run ID, UTC time, command, code commit, dirty-tree flag, configuration hash, dataset hashes, seed, environment, backend, and output checksums.

Verification and acceptance gate. Execute the same experiment twice and verify manifests differ only in permitted runtime fields while configuration, input, and output hashes match.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.20 F0.20: Establish research-data governance

Implementation. Classify raw PCAP, derived counts, models, logs, and reports. Document provenance, license/consent, retention, permitted sharing, anonymization, and deletion procedures. Treat PCAP payloads and endpoint identifiers as sensitive.

Verification and acceptance gate. Complete a data-protection checklist; scan a sample published artifact for payloads, MAC/IP addresses, hostnames, and absolute paths; obtain supervisor approval for the sharing classification.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.21 F0.21: Protect raw evidence

Implementation. Make raw PCAP inputs read-only during experiments; compute SHA-256 checksums; never rewrite them in preprocessing. Store derived data separately with provenance links.

Verification and acceptance gate. Hash every source file before and after preprocessing and assert equality. Attempt an in-place write under the container profile and verify permission denial.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.22 F0.22: Define the security threat model

Implementation. Document assets, trust boundaries, attacker capabilities, poisoning/evasion/replay risks, malicious PCAP parsing, artifact tampering, denial of service, and false-response risks. Map mitigations to tasks.

Verification and acceptance gate. Conduct a STRIDE-style review of the architecture. Every identified high-risk threat must have an owner, mitigation, and verification task or an explicitly accepted residual risk.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.23 F0.23: Write architecture decision records

Implementation. Create ADRs for category exclusivity, silent-window semantics, time-zone handling, likelihood coefficient convention, artifact formats, backend interface, split strategy, and advisory-only responses.

Verification and acceptance gate. Review ADRs against the proposal. Run a traceability check ensuring each listed decision has one accepted ADR and no contradictory configuration default.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.24 F0.24: Define reproducibility commands

Implementation. Add a task runner with single commands for bootstrap, quality checks, fast tests, full tests, preprocessing, experiment reproduction, report generation, and cleanup of disposable outputs.

Verification and acceptance gate. Starting from a clean clone, follow only the README commands to reproduce the smoke experiment. The process must not require undocumented manual file edits.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

2.0.25 F0.25: Create contribution and review policy

Implementation. Document branch naming, atomic commits, issue templates, code review, numerical-review requirements, and the rule that generated results require manifest-backed provenance.

Verification and acceptance gate. Use the template on one pilot change; confirm the review checklist links requirement, test, result, and documentation evidence.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: All F0 tasks pass in CI from a clean clone and the baseline, architecture, threat model, data policy, and ADRs are reviewed.

3 P0: Research Setup and Reproducibility

3.0.1 P0.01: Inventory source captures

Implementation. Enumerate expected and present D-Link Camera5 captures with date, byte size, packet count where readable, checksum, and parser status. Record missing or duplicate files without silently skipping them.

Verification and acceptance gate. Compare inventory entries with the filesystem; assert unique checksums unless duplication is explained; fail the inventory command when a configured required capture is missing.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.2 P0.02: Verify capture integrity

Implementation. Use a PCAP validation tool or complete streaming parse to detect truncation, invalid timestamps, and corrupt records. Record recoverable warnings separately from fatal corruption.

Verification and acceptance gate. Parse every configured source to end-of-file. Store counts and warnings; rerun and confirm identical counts. Quarantine a deliberately truncated fixture and verify correct failure classification.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.3 P0.03: Confirm dataset provenance and benign assumptions

Implementation. Record dataset origin, collection conditions, device identity, known activities, and the evidence supporting “normal” training intervals. Explicitly label uncertainty.

Verification and acceptance gate. Supervisor reviews and signs off the provenance note. Any unverified-benign interval is flagged and excluded or handled by a documented robust-training experiment.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.4 P0.04: Resolve the October 10–11 split decision

Implementation. Reconcile proposal, ALM paper, available captures, and thesis objective; encode the accepted inclusion/exclusion choice in an ADR and configuration.

Verification and acceptance gate. Generate split summaries under both alternatives, document the effect on sample counts, and verify the chosen dates exactly match the approved ADR.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.5 P0.05: Freeze temporal partitions

Implementation. Create disjoint fit, calibration, development-test, and final-test date lists. Preserve chronology and prohibit random window mixing across partitions.

Verification and acceptance gate. Automated tests assert non-overlap, chronological order, capture-level grouping, and that final-test identifiers are unavailable to training and threshold code.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.6 P0.06: Create dataset fingerprints

Implementation. Hash the ordered file list, individual inputs, feature taxonomy, window policy, and partition definition into a dataset version identifier.

Verification and acceptance gate. Changing any file, category order, window length, or date assignment must change the fingerprint; repeating unchanged inputs must reproduce it.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.7 P0.07: Centralize deterministic seeds

Implementation. Define named seeds for simulation, baseline fitting, bootstrap intervals, and randomized algorithms. Propagate generators explicitly instead of using global state.

Verification and acceptance gate. Run the smoke experiment twice in separate processes and compare model parameters, injected anomalies, metrics, and normalized reports exactly or within a declared numerical tolerance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.8 P0.08: Create run-directory isolation

Implementation. Write each experiment to `artifacts/runs/<run-id>/` with subdirectories for manifests, models, thresholds, events, metrics, plots, and logs. Prevent accidental overwrite.

Verification and acceptance gate. Start two concurrent smoke runs; verify no path collision, complete manifests, and atomic finalization. Reusing an existing run ID must fail unless an explicit disposable-run option is used.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.9 P0.09: Separate preprocessing from downstream commands

Implementation. Ensure `train`, `score`, and `forecast` consume versioned processed artifacts and never trigger implicit PCAP preprocessing.

Verification and acceptance gate. Mock the PCAP reader and invoke each downstream command; assert it is never called. Missing processed inputs must produce an actionable error.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.10 P0.10: Build a tiny public smoke fixture

Implementation. Create a payload-free synthetic packet/count fixture small enough for CI, with known timestamps, categories, silent windows, and expected outputs.

Verification and acceptance gate. Verify the fixture contains no real identifiers, its license permits distribution, and its expected preprocessing result is asserted exactly in integration tests.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.11 P0.11: Generate the initial dataset report

Implementation. Report dates, packets, windows, missing intervals, silent intervals, per-category totals, category taxonomy, and partition counts in machine-readable and LaTeX forms.

Verification and acceptance gate. Cross-check totals independently from source or processed files; assert report tables reconcile with dataset manifests and compile the LaTeX fragment.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

3.0.12 P0.12: Create an end-to-end smoke command

Implementation. Provide one command that validates configuration, processes the smoke fixture, diagnoses it, trains, calibrates, scores, and writes a manifest-backed report.

Verification and acceptance gate. Execute from a clean environment in CI; assert successful exit, expected artifact set, valid schemas, checksums, and no modification of inputs.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: The immutable partition and dataset fingerprint are approved, and the deterministic smoke pipeline succeeds twice from clean environments.

4 P1: Data Processing Hardening

4.0.1 P1.01: Define canonical category semantics

Implementation. Choose lowercase canonical names and explicitly decide whether SSDP is exclusive of UDP or counted in both. Record category order and classification priority.

Verification and acceptance gate. Use packets representing TCP, ordinary UDP, SSDP/UDP:1900, ARP, and unknown protocols; assert exactly the counts required by the ADR and fixed column order.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.2 P1.02: Normalize category input

Implementation. Normalize parser output once at the ingestion boundary and reject category collisions after normalization. Remove mixed-case assumptions from later stages.

Verification and acceptance gate. Property-test mixed-case labels and whitespace; verify downstream matrices are identical and duplicate normalized configuration entries fail validation.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.3 P1.03: Harden timestamp parsing

Implementation. Convert all packet times to timezone-aware UTC, reject non-finite values, and preserve sufficient precision. Record original capture time-zone assumptions.

Verification and acceptance gate. Test epoch boundaries, fractional seconds, naive values, out-of-order packets, and daylight-saving examples; assert normalized UTC values.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.4 P1.04: Specify window boundaries

Implementation. Implement half-open windows $[t, t + \Delta)$, with an explicitly chosen origin and deterministic boundary behavior.

Verification and acceptance gate. Place packets immediately before, on, and immediately after a boundary and assert assignment to exactly one correct window for 1, 5, 10, and 30 minutes.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.5 P1.05: Preserve window timestamps

Implementation. Extend `WindowRecord` with device ID, start/end UTC timestamps, ordered counts, total count, missingness state, and metadata.

Verification and acceptance gate. Schema round-trip preserves timestamps and category order; assert $N_t = \sum_{k=1}^K x_{tk}$ for every record.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.6 P1.06: Represent silent windows

Implementation. Materialize elapsed capture windows with zero total count and label them `observed-silent`; do not confuse them with absent capture coverage.

Verification and acceptance gate. Use a fixture with a deliberate quiet interval and verify all expected zero windows exist, remain ordered, and survive JSON/matrix export.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.7 P1.07: Represent missing coverage

Implementation. Define a separate `missing` state for gaps where observation is unavailable; prohibit automatic conversion of missing coverage to zeros.

Verification and acceptance gate. Use a fixture with declared capture discontinuity and assert missing windows are excluded or handled according to policy, never counted as observed silence.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.8 P1.08: Validate count values

Implementation. Reject booleans, negatives, fractional values, NaN, infinity, and integer overflow; use a documented integer width.

Verification and acceptance gate. Parameterized tests exercise every invalid type and maximum supported values. Valid arrays retain exact integer values through all exports.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.9 P1.09: Validate temporal streams

Implementation. Check monotonicity, duplicate windows, overlap, expected duration, device continuity, and category-schema consistency.

Verification and acceptance gate. Inject one defect of each type and assert a specific validation error containing dataset and window identifiers. A valid multi-day stream passes.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.10 P1.10: Make PCAP parsing resource-safe

Implementation. Stream packets with bounded memory, close readers on success and failure, constrain pathological packet handling, and count unsupported packets.

Verification and acceptance gate. Measure memory over a representative capture, inject parser exceptions, and assert file descriptors close and partial outputs are not promoted.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.11 P1.11: Make export atomic

Implementation. Write derived artifacts to a temporary sibling, validate them, fsync where appropriate, and rename atomically; never leave a valid-looking partial file.

Verification and acceptance gate. Interrupt export in a fault-injection test and verify no final artifact exists. A successful export must validate before becoming visible.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.12 P1.12: Export canonical JSON

Implementation. Define canonical JSON with schema version, dataset fingerprint, metadata, window records, and checksums.

Verification and acceptance gate. Round-trip a multi-day fixture and compare every field. Verify stable serialization and checksum across repeated exports.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.13 P1.13: Export modeling matrices

Implementation. Export an efficient tabular or NumPy representation containing timestamps, missingness mask, category columns, totals, device ID, and partition, without losing provenance.

Verification and acceptance gate. Load both canonical JSON and matrix output and assert row count, order, counts, timestamps, and metadata agree exactly.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.14 P1.14: Load matrices by partition

Implementation. Implement a loader that selects only declared partition identifiers and returns $X \in \mathbb{N}^{R \times K}$ plus aligned metadata.

Verification and acceptance gate. Assert shape, dtype, non-negativity, category order, and exact selected dates. Attempt to request an undeclared partition and verify failure.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.15 P1.15: Compute descriptive statistics

Implementation. For each category compute $\bar{x}_{.k}$, sample variance s_k^2 , zero rate, min/max, quantiles, and total counts, with defined behavior for insufficient samples.

Verification and acceptance gate. Compare a hand-calculated fixture against expected values and NumPy/SciPy references within stated tolerance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.16 P1.16: Compute dispersion diagnostics

Implementation. Compute the Poisson dispersion index

$$DI_k = \frac{s_k^2}{\bar{x}_{.k}},$$

returning an explicit undefined state when $\bar{x}_{.k} = 0$.

Verification and acceptance gate. Test underdispersed, equidispersed, overdispersed, and all-zero fixtures. No NaN may enter a serialized report without an explicit status.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.17 P1.17: Report cross-category dependence

Implementation. Compute covariance/correlation where defined and visualize composition dependence to support the Dirichlet–Multinomial rationale.

Verification and acceptance gate. Compare covariance on a known matrix with an independent implementation; mark constant-category correlations undefined rather than silently zero.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.18 P1.18: Generate data-quality reports

Implementation. Produce JSON and LaTeX reports covering validation, missing/silent windows, unknown protocols, dispersion, totals, and partition summaries.

Verification and acceptance gate. Validate JSON against schema, compile LaTeX without warnings treated as errors where feasible, and reconcile every reported count with processed artifacts.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.19 P1.19: Add processing regression fixtures

Implementation. Freeze expected outputs for boundary, casing, SSDP classification, silent, missing, corrupt, and multi-day scenarios.

Verification and acceptance gate. Run regression tests on every supported Python version; any intentional output change must update an ADR and dataset schema/version.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

4.0.20 P1.20: Benchmark preprocessing

Implementation. Measure throughput, peak memory, unsupported-packet rate, and output size on a representative capture without including timing in correctness tests.

Verification and acceptance gate. Repeat at least five times after warm-up; report median and dispersion with host metadata. Confirm counts are identical in all runs.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: All configured captures produce schema-valid, time-indexed, checksum-backed matrices, and processing reports reconcile with source inventories.

5 P2: Dirichlet–Multinomial Modeling

5.0.1 P2.01: Define estimator input contract

Implementation. Accept only two-dimensional non-negative integer matrices with $R > 1$, $K > 1$, stable category order, and a documented policy for zero-total rows and all-zero categories.

Verification and acceptance gate. Parameterized tests reject every invalid dimension and policy violation; a valid matrix is not mutated by training.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.2 P2.02: Verify the likelihood formula

Implementation. Implement the reference log probability

$$\log p(\mathbf{x} \mid \boldsymbol{\alpha}) = \log \Gamma(N + 1) - \sum_{k=1}^K \log \Gamma(x_k + 1) \quad (1)$$

$$+ \log \Gamma(\alpha_0) - \log \Gamma(N + \alpha_0) + \sum_{k=1}^K [\log \Gamma(x_k + \alpha_k) - \log \Gamma(\alpha_k)], \quad (2)$$

where $\alpha_0 = \sum_k \alpha_k$. Make inclusion of the multinomial coefficient explicit.

Verification and acceptance gate. Compare small cases with direct arbitrary-precision probability enumeration and verify the coefficient convention with a hand-calculated example.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.3 P2.03: Implement robust initialization

Implementation. Provide mean-concentration and moment-based initializers, floors for sparse categories, and deterministic fallback behavior.

Verification and acceptance gate. Test balanced, highly skewed, sparse, and nearly constant synthetic matrices; every returned α_k must be finite and strictly positive.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.4 P2.04: Harden the fixed-point update

Implementation. Separate one Minka update from the iteration loop, validate numerator and denominator, guard positivity, and detect non-finite or zero updates.

Verification and acceptance gate. Unit-test the update against a stored high-precision example and force each guard branch with synthetic pathological inputs.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.5 P2.05: Define convergence criteria

Implementation. Require both parameter stability and likelihood stability, use absolute plus relative tolerances, impose maximum iterations, and distinguish converged, stalled, diverged, and exhausted outcomes.

Verification and acceptance gate. Construct fixtures for each outcome and assert the status, iteration count, and diagnostic reason. Never label maximum-iteration exhaustion as convergence.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.6 P2.06: Record optimization history

Implementation. Store iteration, α , log-likelihood, absolute/relative change, update norm, and elapsed time with configurable bounded retention.

Verification and acceptance gate. Verify history length and final entry agree with the returned model. Serialized history must reload without loss beyond declared float precision.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.7 P2.07: Activate the modeling stage

Implementation. Load only the fit partition, initialize and fit the estimator, derive α_0 , $p_k = \alpha_k/\alpha_0$, and $\psi = 1/\alpha_0$, then persist diagnostics.

Verification and acceptance gate. Run the CLI on the smoke matrix; assert a valid model artifact, successful exit, correct categories, positive alpha, $\sum_k p_k = 1$, and no access to calibration/test partitions.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.8 P2.08: Define the model artifact

Implementation. Include schema version, model version, categories, parameters, training range, dataset/config/code hashes, backend, convergence, runtime, and integrity checksum.

Verification and acceptance gate. Validate against schema, round-trip exactly, reject category reordering and checksum corruption, and verify no raw packet data is embedded.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.9 P2.09: Recover synthetic parameters

Implementation. Sample multiple seeded datasets from known α^* , fit them, and quantify error in mean composition and concentration.

Verification and acceptance gate. Across increasing sample sizes, demonstrate decreasing median estimation error or explain deviations; enforce predeclared tolerances on a stable medium-size fixture.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.10 P2.10: Test permutation equivariance

Implementation. Permute category columns and initialization consistently; fitted parameters should permute in the same way.

Verification and acceptance gate. Property-test random permutations and compare unpermuted results within numerical tolerance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.11 P2.11: Test row-order invariance

Implementation. Shuffle training windows while preserving values; batch MLE results should remain unchanged up to floating-point tolerance.

Verification and acceptance gate. Run at least ten seeded shuffles and compare parameters and terminal likelihood with declared tolerances.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.12 P2.12: Assess likelihood stability

Implementation. Check for monotonic improvement where the selected fixed-point method guarantees it; otherwise define and test an accepted small numerical-decrease tolerance and stalled-run policy.

Verification and acceptance gate. Inspect synthetic and real histories automatically. Any decrease beyond tolerance fails training and preserves diagnostics for analysis.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.13 P2.13: Create model-fit diagnostics

Implementation. Report observed versus fitted mean composition, concentration, dispersion, per-window likelihood distribution, and residual summaries.

Verification and acceptance gate. On simulated data, verify fitted summaries approach generating values; compile JSON and LaTeX reports and reconcile model identifiers.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

5.0.14 P2.14: Benchmark estimator scaling

Implementation. Measure iterations, wall time, and peak memory as R , K , counts, and overdispersion vary.

Verification and acceptance gate. Use a parameterized benchmark matrix, repeat runs, and store median/quantiles with environment metadata and correctness checks.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: Training is partition-safe, produces an integrity-checked converged model, passes recovery/invariance tests, and has a reviewed fit report.

6 P3: Likelihood Backend Abstraction

6.0.1 P3.01: Define the backend protocol

Implementation. Create a stateless `LikelihoodBackend` interface returning value, precision status, error bound when available, backend version, and timing metadata; support single-window and batch calls.

Verification and acceptance gate. Contract-test a fake backend and every real backend for shape, dtype, metadata, immutability, error semantics, and consistent coefficient convention.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.2 P3.02: Implement the standard `gammaln` backend

Implementation. Move SciPy-based likelihood evaluation behind the interface and support stable vectorized batch evaluation.

Verification and acceptance gate. Compare with the P2 arbitrary-precision oracle across small random cases and with the previous implementation on regression fixtures.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.3 P3.03: Build a high-precision oracle

Implementation. Use `mpmath` at configurable precision to evaluate selected log probabilities and paired log-gamma differences outside production code.

Verification and acceptance gate. Validate oracle precision by recomputing at doubled digits; accepted reference digits must remain unchanged. Store reference vectors and provenance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.4 P3.04: Specify the LM numerical contract

Implementation. From the cited LM algorithm, document domain, requested precision, truncation/error rules, Bernoulli constants, fallback behavior, and statuses `ok`, `warning`, and `precision-impossible`.

Verification and acceptance gate. Review the specification line-by-line against the source paper and reproduce at least one published numerical example before implementation acceptance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.5 P3.05: Implement paired log-gamma differences

Implementation. Implement the LM primitive for expressions such as $\log \Gamma(x + a) - \log \Gamma(a)$, with no silent fallback and explicit error estimates.

Verification and acceptance gate. Compare a logarithmic grid of small/large x , small/large a , and low-overdispersion cases with the doubled-precision oracle. Enforce requested error bounds.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.6 P3.06: Assemble the Python LM backend

Implementation. Use the LM primitive for all paired terms in the Dirichlet–Multinomial likelihood and retain an exact/reference path for factorial terms.

Verification and acceptance gate. Contract-test against standard and oracle backends over randomized and adversarial grids. Assert correct precision status propagation.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.7 P3.07: Add optional Yu–Shaw comparison

Implementation. Implement YS only behind the same interface, using the literature-defined method and documented feasible domain; otherwise record a justified scope exclusion.

Verification and acceptance gate. Reproduce a cited example and compare accuracy/runtime with the oracle. Unsupported cases must return a typed status, not a plausible number.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.8 P3.08: Test extreme numerical regimes

Implementation. Cover zero counts, huge counts, highly imbalanced categories, α_k near the lower bound, very large concentration, and cancellation-prone low-overdispersion cases.

Verification and acceptance gate. No accepted result may be NaN or infinite. Compare to the oracle and enforce absolute/relative/ULP tolerances appropriate to each regime.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.9 P3.09: Test additive batch consistency

Implementation. The sum of single-window log probabilities must agree with the batch log-likelihood under the same coefficient convention.

Verification and acceptance gate. Property-test random matrices and all backends; compare with a scale-aware floating-point tolerance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.10 P3.10: Implement backend selection

Implementation. Select backend only through validated configuration and record backend name/version in every model, score, and manifest.

Verification and acceptance gate. Run identical scoring with each enabled backend and verify artifact metadata. Invalid or unavailable selections must fail before data processing.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.11 P3.11: Create backend benchmarks

Implementation. Benchmark accuracy, error status, latency, throughput, and memory across counts, dimensions, concentration, and batch sizes. Separate cold-start from steady-state results.

Verification and acceptance gate. Repeat randomized benchmark order, report median and tail latency, validate every benchmark output against the oracle subset, and archive host/toolchain details.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

6.0.12 P3.12: Prototype the optimized LM binding

Implementation. If justified by benchmarks, expose a C/C++ LM kernel through a narrow, memory-safe binding with explicit array ownership, bounds checking, and compiler flags.

Verification and acceptance gate. Use sanitizers in native tests, fuzz invalid shapes/values, compare results with Python LM/oracle, and verify no leak across repeated calls.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: Every enabled backend satisfies one interface, numerical errors are bounded against an independent oracle, and benchmark claims are reproducible.

7 P4: Threshold Calibration and Anomaly Scoring

7.0.1 P4.01: Enforce fit/calibration isolation

Implementation. Load calibration windows through a partition-scoped API that cannot return fit or final-test rows.

Verification and acceptance gate. Assert disjoint identifiers and monkeypatch the fit loader to fail if calibration invokes it. Store partition fingerprints in the threshold artifact.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.2 P4.02: Compute raw window scores

Implementation. Return one log probability $s_t = \log p(\mathbf{x}_t \mid \hat{\alpha})$ and precision metadata per eligible window.

Verification and acceptance gate. Compare each score with the backend single-window call; preserve timestamps/order and explicitly classify missing windows.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.3 P4.03: Define normalized scores

Implementation. Implement a documented normalization, such as per-packet score $s_t / \max(N_t, 1)$, while treating silent windows according to ADR rather than dividing silently.

Verification and acceptance gate. Test totals $N_t = 0, 1$, and large N_t ; assert finite values or explicit non-applicability and record the formula in artifacts.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.4 P4.04: Analyze volume confounding

Implementation. Measure the relationship between raw score and total count to decide whether raw, normalized, stratified, or paired thresholds are primary.

Verification and acceptance gate. Produce scatter/correlation and binned false-alarm analyses on calibration only. Record the selected policy in an ADR before final-test access.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.5 P4.05: Calibrate empirical quantiles

Implementation. Compute the lower-tail threshold $\tau = Q_q(\{s_i\}_{i=1}^m)$ with declared quantile convention and target calibration false-positive rate.

Verification and acceptance gate. Compare with an independent quantile implementation, test tiny/tied samples, and verify observed calibration alarm rate matches the convention within finite-sample limits.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.6 P4.06: Quantify threshold uncertainty

Implementation. Bootstrap calibration windows in time-aware blocks and compute a confidence interval for τ without touching test data.

Verification and acceptance gate. With fixed seeds, reproduce interval endpoints; verify block resampling preserves required temporal dependence and report the effective sample size.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.7 P4.07: Persist threshold artifacts

Implementation. Store model checksum, score type, quantile, threshold, uncertainty, calibration range/fingerprint, sample count, backend, and schema version.

Verification and acceptance gate. Round-trip and checksum-test the artifact. Scoring must reject a threshold created for another model, backend convention, or category schema.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.8 P4.08: Implement online scoring

Implementation. Consume windows in timestamp order and emit score records without future context. Keep state bounded and deterministic.

Verification and acceptance gate. Replay the same stream in different chunk sizes and assert identical ordered outputs; monitor memory over a long synthetic stream.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.9 P4.09: Implement anomaly decisions

Implementation. Apply the strict documented rule $s_t < \tau$ (including explicit equality behavior) and retain both score and threshold.

Verification and acceptance gate. Test values below, equal to, and above threshold, including floating-point neighbors. Decisions must be backend-metadata aware.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.10 P4.10: Calibrate severity

Implementation. Use a robust calibration scale such as IQR or median absolute deviation and define bounded severity bands from distance below threshold.

Verification and acceptance gate. Test degenerate calibration distributions, monotonicity, band boundaries, and invariance to irrelevant record ordering.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.11 P4.11: Compute category residual explanations

Implementation. For $N > 0$, calculate

$$r_k = \frac{x_k}{N} - \frac{\alpha_k}{\alpha_0}, \quad e_k = N \frac{\alpha_k}{\alpha_0},$$

and rank positive and negative deviations; use a separate explanation for silence.

Verification and acceptance gate. Hand-check a known vector, assert residuals sum to zero within tolerance, category ordering is deterministic under ties, and silent windows never divide by zero.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.12 P4.12: Validate explanation faithfulness

Implementation. Distinguish residual heuristics from exact additive likelihood attribution and label them accurately in output and thesis text.

Verification and acceptance gate. Inject one-category increases/decreases at fixed total and verify the expected category ranks first in a predefined majority of controlled cases; document known counterexamples.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.13 P4.13: Define anomaly event schemas

Implementation. Create a versioned event containing device/window IDs, counts, score, threshold, decision, severity, explanation, precision status, model/threshold versions, and run ID.

Verification and acceptance gate. Schema-test mandatory fields, reject incompatible category order, round-trip JSON/CSV, and verify no packet payload or unnecessary endpoint identifier is present.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.14 P4.14: Make event output durable

Implementation. Write events atomically, support idempotent event IDs, and prevent duplicates during replay/restart.

Verification and acceptance gate. Interrupt and resume a stream; compare output with uninterrupted execution and assert unique event IDs and identical decisions.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

7.0.15 P4.15: Generate calibration reports

Implementation. Plot/table score distributions, threshold, uncertainty, target/observed alarm rate, score-volume behavior, and examples without using final-test labels.

Verification and acceptance gate. Regenerate from artifacts alone, compile the LaTeX fragment, and verify all figures cite model, threshold, dataset, and run identifiers.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: A model-bound threshold produces deterministic streaming decisions and schema-valid, interpretable events with documented calibration uncertainty and no data leakage.

8 P5: Experimental Anomaly Evaluation

8.0.1 P5.01: Audit labels and attack data

Implementation. Inventory available labels, their provenance, temporal precision, attack taxonomy, and ambiguity. Do not infer ground truth from detector output.

Verification and acceptance gate. Cross-check labels with source documentation and sample records; report unlabeled and conflicting intervals explicitly.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.2 P5.02: Pre-register the evaluation protocol

Implementation. Freeze hypotheses, primary score, metrics, baselines, anomaly strengths, seeds, exclusion rules, and statistical tests before final evaluation.

Verification and acceptance gate. Hash and commit the protocol; evaluation code verifies the protocol hash and refuses undeclared final-test parameter tuning.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.3 P5.03: Define synthetic injection semantics

Implementation. Inject anomalies only into copies of held-out benign count windows; preserve timestamps, originals, labels, seed, changed categories, magnitude, and feasibility constraints.

Verification and acceptance gate. For every injected row, verify an untouched source row exists and a machine-readable diff matches the declared transformation exactly.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.4 P5.04: Implement TCP flood injections

Implementation. Increase TCP counts using absolute and multiplicative severity levels with overflow protection and fixed/variable-total variants if scientifically justified.

Verification and acceptance gate. Property-test non-target categories, totals, labels, determinism, and monotonic injected severity. Inspect representative cases.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.5 P5.05: Implement UDP burst injections

Implementation. Apply the approved UDP transformation while respecting the SSDP exclusivity ADR.

Verification and acceptance gate. Assert SSDP is not accidentally changed, metadata is complete, and replay with the same seed is identical.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.6 P5.06: Implement SSDP discovery injections

Implementation. Increase SSDP counts according to the taxonomy and category exclusivity policy.

Verification and acceptance gate. Verify generated counts are internally consistent with the chosen UDP/SSDP semantics and increasing severity yields non-decreasing SSDP perturbation.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.7 P5.07: Implement ARP scan injections

Implementation. Create temporally localized or repeated ARP count increases representing the stated abstract scan model, without claiming packet-level realism not present in counts.

Verification and acceptance gate. Verify duration, affected windows, magnitude, and labels exactly; document the construct-validity limitation.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.8 P5.08: Implement cloud-traffic suppression

Implementation. Reduce selected TCP/traffic components using bounded factors without negative counts, with an explicit rationale for the cloud proxy.

Verification and acceptance gate. Assert counts remain non-negative integers, reductions match factors after rounding, and zero-floor behavior is tested.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.9 P5.09: Implement mixed composition drift

Implementation. Shift mass among categories under fixed-total and variable-total variants, preserving integer counts and declared severity.

Verification and acceptance gate. Assert fixed-total variants conserve N , proportions move in the intended direction, and all changes are logged.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.10 P5.10: Validate anomaly generators

Implementation. Review statistical magnitude and domain plausibility; report perturbations relative to benign quantiles and avoid equating synthetic anomalies with real attacks.

Verification and acceptance gate. Generate a validation report with before/after distributions and have the thesis supervisor approve the terminology and severity grid.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.11 P5.11: Implement independent Poisson baseline

Implementation. Fit one benign-rate model per category using fit data and calibrate its anomaly score on the same calibration partition.

Verification and acceptance gate. Recover known rates on synthetic data, test zero-rate handling, and verify no test labels affect fitting or calibration.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.12 P5.12: Implement Multinomial baseline

Implementation. Fit fixed composition probabilities and score counts under the same coefficient convention and partition controls.

Verification and acceptance gate. Compare small probabilities with a trusted reference and verify fitted probabilities sum to one.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.13 P5.13: Implement Gaussian baseline

Implementation. Define transformations, regularization, and singular-covariance behavior for count/proportion features.

Verification and acceptance gate. Recover a nonsingular synthetic Gaussian case, exercise singular data, and ensure all scores are finite or explicitly rejected.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.14 P5.14: Implement Isolation Forest baseline

Implementation. Use a fixed seed and a predeclared feature pipeline; tune contamination or threshold only on calibration data.

Verification and acceptance gate. Repeat training for deterministic scores within library guarantees, inspect feature shapes, and prove final-test labels are inaccessible.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.15 P5.15: Implement One-Class SVM baseline

Implementation. Scale features using fit-only statistics and select hyperparameters from a predefined calibration grid.

Verification and acceptance gate. Test scaler isolation, deterministic inference, and record the entire selection table rather than only the winner.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.16 P5.16: Standardize baseline interfaces

Implementation. Require all detectors to implement fit, calibrate, score, save/load, metadata, and explanation-capability declarations.

Verification and acceptance gate. Run one contract suite against every detector; compare row IDs to ensure identical evaluation populations.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.17 P5.17: Implement window-level metrics

Implementation. Compute confusion matrix, TPR/recall, FPR, precision, specificity, F1, balanced accuracy, ROC-AUC, and PR-AUC with explicit zero-denominator behavior.

Verification and acceptance gate. Compare known confusion matrices with a trusted metrics library and test single-class edge cases.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.18 P5.18: Implement event-level metrics

Implementation. Merge contiguous labeled anomalous windows according to a fixed rule and compute event recall, false alerts per day, and detection delay.

Verification and acceptance gate. Use hand-labeled timelines covering immediate, delayed, missed, duplicated, and pre-event alarms; assert exact expected results.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.19 P5.19: Estimate uncertainty

Implementation. Use device/day-aware or temporal-block bootstrap confidence intervals and paired resampling for detector differences.

Verification and acceptance gate. Simulate a known metric distribution, verify reproducibility and sensible coverage, and report resampling units and replicate count.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.20 P5.20: Run sensitivity analysis

Implementation. Evaluate predeclared window lengths, category sets, threshold quantiles, score normalization, and anomaly strengths without selecting on final-test performance.

Verification and acceptance gate. Each run must have a unique manifest; aggregate only compatible runs and show all planned settings, including failures.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.21 P5.21: Run ablation studies

Implementation. Remove LM backend, overdispersion modeling, normalization, explanation, and later adaptation components one at a time where meaningful.

Verification and acceptance gate. Verify each ablation changes only the named component by comparing normalized configuration/manifests.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.22 P5.22: Perform final locked evaluation

Implementation. Execute the pre-registered protocol once on the untouched final partition and store immutable outputs.

Verification and acceptance gate. Confirm clean tree, protocol hash, dataset fingerprint, and CI status before execution. Seal outputs with checksums and prohibit post-hoc overwriting.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.23 P5.23: Analyze false positives and negatives

Implementation. Sample errors by day, volume, silence/missingness, category residual, and behavioral context; distinguish model, threshold, data, and label failures.

Verification and acceptance gate. Use a reproducible sampling query and account for every sampled row. Record findings without changing the locked primary result.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

8.0.24 P5.24: Generate publication tables and figures

Implementation. Produce manifest-backed LaTeX tables for primary metrics, confidence intervals, baseline comparisons, sensitivity, and errors; generate accessible vector plots.

Verification and acceptance gate. Rebuild all outputs from result artifacts, compile them in a minimal LaTeX document, and cross-check selected cells against raw metrics.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: The locked protocol has run on held-out data, all baselines share partitions and metrics, uncertainty is reported, and synthetic-result claims are explicitly bounded.

9 P6: Adaptive Retraining

9.0.1 P6.01: Define adaptation states

Implementation. Model states such as collecting, eligible, candidate-training, shadow-evaluation, promoted, rejected, and rolled-back with explicit transitions.

Verification and acceptance gate. State-machine tests exercise every legal transition and reject illegal transitions, retries, and duplicate promotion events.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.2 P6.02: Implement the safe-window predicate

Implementation. Use

$$\text{safe}(t) \iff s_t \geq \tau + m$$

with configurable margin m , valid precision status, acceptable missingness, and any cooldown rules.

Verification and acceptance gate. Test exact boundaries, invalid precision, silent/missing windows, and anomaly cooldown periods.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.3 P6.03: Implement a bounded safe buffer

Implementation. Store only eligible records with IDs, scores, timestamps, model version, and integrity metadata; cap by age and count.

Verification and acceptance gate. Stream more than capacity, restart, and verify deterministic eviction, persistence, ordering, deduplication, and bounded storage.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.4 P6.04: Trigger candidate training

Implementation. Trigger only after predeclared safe-window count/time conditions and prevent overlapping updates for the same device.

Verification and acceptance gate. Use a simulated clock to test just-before/at/after boundaries and concurrent trigger attempts.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.5 P6.05: Train candidates in isolation

Implementation. Fit a candidate from the approved adaptation window without mutating the active model or threshold.

Verification and acceptance gate. Inject training failure and verify active artifacts remain byte-identical; successful candidates receive new immutable versions.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.6 P6.06: Limit parameter movement

Implementation. Constrain change in composition, concentration, or both; optionally use

$$\alpha_a = (1 - \rho)\alpha_o + \rho\alpha_n, \quad 0 < \rho \leq 1.$$

Verification and acceptance gate. Test ρ boundaries, positivity, deterministic output, and rejection of candidates exceeding configured drift limits.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.7 P6.07: Recalibrate candidate thresholds safely

Implementation. Use a candidate-specific benign validation buffer disjoint from candidate fitting data, or retain the old threshold under a documented policy.

Verification and acceptance gate. Assert model-threshold binding and partition disjointness; reject insufficient calibration samples.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.8 P6.08: Evaluate candidates in shadow mode

Implementation. Score a fixed recent validation horizon with old and candidate models without affecting alerts; compare false-alarm proxy, likelihood stability, and resource cost.

Verification and acceptance gate. Replay a deterministic horizon and assert promotion metrics are identical across runs and contain both model versions.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.9 P6.09: Define promotion policy

Implementation. Predeclare minimum improvement/non-inferiority, maximum parameter drift, convergence, numerical status, and resource limits. Require every condition.

Verification and acceptance gate. Boundary-test each criterion and prove a candidate failing any single condition is rejected with an auditable reason.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.10 P6.10: Promote atomically

Implementation. Switch active model and threshold as one transaction while retaining the previous pair and audit record.

Verification and acceptance gate. Fault-inject between write steps and verify readers observe either the complete old pair or complete new pair, never a mixed pair.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.11 P6.11: Implement automatic rollback

Implementation. Rollback when post-promotion alarm-rate, numerical, integrity, or operational guardrails fail within an observation window.

Verification and acceptance gate. Simulate each guardrail violation and verify atomic restoration, incident record, and cooldown against immediate re-promotion.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.12 P6.12: Retain model lineage

Implementation. Record parent model, safe-buffer fingerprint, training/calibration ranges, policy version, evaluation, promotion actor, and rollback reason.

Verification and acceptance gate. Traverse lineage from current to initial model and verify checksums and absence of cycles or missing parents.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.13 P6.13: Simulate gradual benign drift

Implementation. Generate or select chronologically shifting benign compositions and compare static versus adaptive false alarms and fit.

Verification and acceptance gate. Verify the drift generator independently, repeat across seeds, and report benefit with uncertainty and adaptation lag.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.14 P6.14: Simulate abrupt benign change

Implementation. Evaluate firmware-like regime change without labeling it an attack and measure recovery time and transient alarms.

Verification and acceptance gate. Confirm change point and magnitude; calculate recovery from a predeclared rule and compare policies.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.15 P6.15: Test poisoning resistance

Implementation. Inject low-and-slow, margin-crossing, burst, and mixed-category contamination into the update stream; measure parameter displacement and attack acceptance.

Verification and acceptance gate. Compare clean and poisoned lineage under identical seeds. Enforce a declared maximum movement or clearly report policy failure.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.16 P6.16: Test evasion near threshold

Implementation. Construct perturbations that remain just inside the safe margin and quantify buffer admission and later detection impact.

Verification and acceptance gate. Verify score placement with the active backend and document whether attacks are truly feasible at the packet level or only count-space stress tests.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.17 P6.17: Compare adaptation policies

Implementation. Evaluate static, full replacement, smoothing rates, buffer sizes, margins, and update frequencies under the frozen protocol.

Verification and acceptance gate. Use identical streams and paired uncertainty; do not choose a policy on final-test results.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

9.0.18 P6.18: Generate the adaptation audit report

Implementation. Report lineage, promotions/rejections, rollback events, drift response, poisoning sensitivity, compute cost, and residual risks.

Verification and acceptance gate. Rebuild report from audit events and artifacts only; reconcile every model version and policy decision.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: Adaptation remains off by default until shadow, atomic promotion, rollback, drift, and poisoning tests pass; the static detector remains a supported fallback.

10 P7: Forecasting and Response Support

10.0.1 P7.01: Remove synthetic forecasting actuals

Implementation. Load chronologically held-out real window counts and preserve missing/silent semantics; prohibit sampling “actuals” from the fitted model.

Verification and acceptance gate. Mock random sampling to fail during production forecasting and verify returned observations match processed artifact IDs and counts exactly.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.2 P7.02: Define composition forecasts

Implementation. Forecast $\hat{p}_{t+1,k} = \alpha_k/\alpha_0$ with the model version valid at forecast origin and prevent future leakage.

Verification and acceptance gate. For every forecast, verify probabilities are non-negative, sum to one, and cite an origin preceding the target window.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.3 P7.03: Implement moving-average total forecasts

Implementation. Predict N_{t+h} using only a predeclared trailing horizon and define behavior for missing history.

Verification and acceptance gate. Hand-check a short sequence and use a future-value sentinel to prove no leakage.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.4 P7.04: Implement seasonal total forecasts

Implementation. Create time-of-day/day-of-week benign baselines with fallback for sparse seasonal cells.

Verification and acceptance gate. Test calendar boundaries and unavailable cells; compare forecasts with independently grouped training data.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.5 P7.05: Combine total and composition

Implementation. Compute expected counts $\hat{x}_{t+h,k} = \hat{N}_{t+h}\hat{p}_{t+h,k}$ while retaining probabilistic composition output.

Verification and acceptance gate. Assert category expected counts sum to predicted total within tolerance and remain non-negative.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.6 P7.06: Correct the Brier metric contract

Implementation. Define whether Brier score assesses multiclass probability outcomes or distribution forecasts; do not call squared distance to observed proportions a standard Brier score without qualification.

Verification and acceptance gate. Validate the implementation on canonical one-hot examples and document any generalized distribution-score variant separately.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.7 P7.07: Harden Jensen–Shannon divergence

Implementation. Normalize inputs, define zero handling, select logarithm base, and compute per-window rather than accidentally aggregating across time.

Verification and acceptance gate. Test identity, symmetry, finiteness, and the theoretical bound for the selected log base against trusted examples.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.8 P7.08: Add total-count forecast metrics

Implementation. Implement MAE, RMSE, and optionally Poisson deviance with missing-value policy.

Verification and acceptance gate. Compare known vectors with a trusted library or hand calculation and test all-zero targets.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.9 P7.09: Use rolling-origin evaluation

Implementation. Evaluate forecasts through chronological origins with fixed horizons and no refitting on future targets.

Verification and acceptance gate. Inspect generated origin/target pairs; automated checks require origin time less than every target time.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.10 P7.10: Report forecast uncertainty

Implementation. Use empirical or model-based intervals with documented assumptions and measure interval coverage.

Verification and acceptance gate. On seeded synthetic data, test monotonic interval endpoints and approximate expected coverage before real-data use.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.11 P7.11: Define advisory response taxonomy

Implementation. Create versioned response codes for monitor, operator alert, rate-limit recommendation, isolation recommendation, SSDP inspection, ARP inspection, and connectivity investigation, including reversibility, cost, and approval requirement.

Verification and acceptance gate. Schema-test all response records and obtain security/supervisor review that wording is advisory and avoids unsupported certainty.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.12 P7.12: Implement deterministic response rules

Implementation. Map severity, repetition, category residual pattern, confidence, and device criticality to recommendations using configuration rather than scattered conditionals.

Verification and acceptance gate. Table-test every rule, boundary, tie, and default. Same event and policy version must always yield the same recommendation.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.13 P7.13: Enforce advisory-only operation

Implementation. Provide no packet-blocking, firewall-changing, or device-isolation executor in the thesis prototype; mark human approval explicitly.

Verification and acceptance gate. Search code and container permissions for mutation paths; integration-test that response generation writes records only and performs no network-control action.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

10.0.14 P7.14: Audit response provenance

Implementation. Link each recommendation to anomaly event, explanation, policy version, model, and approval state.

Verification and acceptance gate. Trace a sample response back to checksummed source artifacts and verify policy replay reproduces it.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.15 P7.15: Generate forecasting and response reports

Implementation. Report composition and total metrics by horizon, baseline comparisons, uncertainty, recommendation counts, and limitations.

Verification and acceptance gate. Rebuild from artifacts, compile LaTeX, and cross-check sampled rows and recommendation totals.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: Forecasts use only real held-out observations and past context, metric names are statistically correct, and response output is deterministic, auditable, and strictly advisory.

11 P8: Edge Deployment Study

11.0.1 P8.01: Define target edge profiles

Implementation. Specify representative CPU architecture, cores, memory, storage, OS, power assumptions, window arrival rate, and latency/service-level objectives.

Verification and acceptance gate. Review profiles against actual accessible hardware or clearly label emulation; freeze benchmark profiles before comparison.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.2 P8.02: Create a benchmark harness

Implementation. Measure training and per-window/batch scoring latency, throughput, peak RSS, CPU time, artifact size, startup, and optional energy, separating I/O from numerical kernels.

Verification and acceptance gate. Validate timers with a known-duration operation, include warm-up, repeat runs, and store raw samples and host metadata.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.3 P8.03: Benchmark standard Python backend

Implementation. Run the fixed workload and model under the standard gammaln backend.

Verification and acceptance gate. Verify output checksums against desktop reference, report median, p95/p99, dispersion, and resource limits across repeated runs.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.4 P8.04: Benchmark Python LM backend

Implementation. Run the identical workload under Python LM without changing model/data/configuration beyond backend.

Verification and acceptance gate. Manifest diff must show only the backend; numerical deviations must remain inside P3 tolerances.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.5 P8.05: Benchmark optimized LM backend

Implementation. Where implemented, compile reproducibly for target architecture and benchmark the same workload.

Verification and acceptance gate. Verify compiler/toolchain manifest, numerical equivalence, sanitizer tests, and repeated performance samples.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.6 P8.06: Measure bounded-memory streaming

Implementation. Replay a long stream and confirm memory stabilizes under configured buffers, logging, and adaptation settings.

Verification and acceptance gate. Plot RSS over time and fail if post-warm-up growth exceeds a predeclared bound after accounting for caches.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.7 P8.07: Measure deadline compliance

Implementation. Compare end-to-end processing and scoring latency with the configured window interval and burst arrival scenarios.

Verification and acceptance gate. Report deadline miss ratio and p99 latency; demonstrate backlog recovery or document infeasibility.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.8 P8.08: Apply least privilege

Implementation. Run as a non-root user with read-only raw/config/model mounts where possible, a dedicated writable artifact directory, dropped Linux capabilities, no-new-privileges, and no exposed port unless required.

Verification and acceptance gate. Inspect runtime identity/capabilities and attempt prohibited writes and privilege escalation; all must fail while scoring succeeds.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.9 P8.09: Secure the software supply chain

Implementation. Pin base image digest and dependencies, generate an SBOM, scan dependencies/images, document licenses, and define handling for unresolved vulnerabilities.

Verification and acceptance gate. CI stores SBOM and scan reports; builds fail on unwaived critical vulnerabilities and every waiver has expiry and rationale.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.10 P8.10: Verify offline operation

Implementation. Ensure inference requires no Internet access, telemetry, dependency download, or external service after image construction.

Verification and acceptance gate. Run the complete scoring smoke test with networking disabled and compare outputs with the connected run.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.11 P8.11: Test artifact integrity at deployment

Implementation. Verify model and threshold checksums, schema compatibility, category order, and policy version before startup; fail closed on mismatch.

Verification and acceptance gate. Corrupt and swap each artifact independently and verify startup rejection and a structured audit event.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.12 P8.12: Test graceful restart and replay

Implementation. Persist stream checkpoint and idempotent event IDs so restart neither loses nor duplicates completed-window decisions.

Verification and acceptance gate. Terminate at multiple processing points, restart, and compare output exactly with uninterrupted reference execution.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.13 P8.13: Test resource exhaustion behavior

Implementation. Apply CPU/memory/disk constraints and oversized/malformed inputs; bound queues and fail safely without corrupting artifacts.

Verification and acceptance gate. Run under container limits, fill the artifact volume in a controlled temporary environment, and verify explicit errors and intact previous artifacts.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.14 P8.14: Document deployment

Implementation. Write build, configuration, mounting, execution, health, logging, upgrade, rollback, backup, and incident procedures plus privacy warnings for PCAP data.

Verification and acceptance gate. A second person follows the guide on a clean host and completes offline smoke scoring and rollback without undocumented assistance.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

11.0.15 P8.15: Generate the edge-readiness report

Implementation. Compare backends and target profiles on correctness, latency, memory, CPU, model size, startup, and security posture; state whether near-real-time objectives are met.

Verification and acceptance gate. Rebuild all tables/plots from raw benchmark artifacts and manifests; independently verify selected values and confidence summaries.

Status. ☐ Not started ☐ In progress ☐ Blocked ☐ Done

Phase exit gate: A pinned, non-root, offline-capable image passes correctness, integrity, restart, resource-limit, vulnerability, and reproducible performance tests on the declared target profile.

12 Cross-Phase Research and Release Controls

12.1 Experiment Validity Checklist

Before any result enters the thesis, verify:

- the research question and metric were declared before final-test access;
- fit, calibration, development, and final-test partitions are disjoint and chronological;
- preprocessing, category semantics, and missing/silent-window policy are identical across compared methods;
- baselines receive the same information and an equitable calibration budget;
- random seeds, negative results, exclusions, and failed runs are retained;
- uncertainty accompanies point estimates and the resampling unit respects temporal/device dependence;
- synthetic anomalies are called injections or stress tests, not evidence of detection of real attacks;
- plots and tables can be regenerated from immutable artifacts without hand-edited values.

12.2 Numerical Reliability Checklist

For every likelihood backend and estimator release:

- validate $\alpha_k > 0$, $x_k \in \mathbb{N}_0$, and compatible dimensions before computation;
- compare selected cases with an independent arbitrary-precision oracle;
- exercise cancellation-prone low-overdispersion and high-count regimes;
- expose, propagate, and archive precision status and error bounds;
- prohibit silent backend fallback or conversion of invalid output into a normal score;
- state whether the multinomial coefficient is included in every score and comparison.

12.3 Release Checklist

Tag a thesis release only when the tree is clean; CI and container scans pass; configuration, dataset, protocol, and code hashes are recorded; model and result artifacts validate; documentation compiles; licenses and data-sharing rules are satisfied; and a clean-environment reproduction has succeeded. The release notes must list limitations, unsupported environments, known numerical regimes, and any deviation from this roadmap.

13 Optional Extensions After the Core Gates

Theoretical extensions from Chapter 5 are deliberately deferred until P0–P5 are complete. Each extension begins with a separate hypothesis and baseline: latent traffic-topic modes; hierarchical device models; contextual parameters by time/device state; zero-inflated silence modeling; Bayesian threshold uncertainty; formal influence-bounded adaptation; or a Dirichlet–Softmax bridge. No extension may change the locked primary experiment retroactively.

14 Final Deliverable Matrix

Deliverable	Minimum contents	Acceptance owner
Reproducible codebase	Typed package, stable CLI, tests, CI, locked environment, security checks	Engineer/reviewer
Processed dataset	Provenance, checksum, schema, timestamps, partitions, silent/missing states	Researcher/supervisor

Deliverable	Minimum contents	Acceptance owner
Normal model	Alpha, mean, concentration, overdispersion, backend, convergence, lineage	Numerical reviewer
Threshold model	Model binding, calibration fingerprint, uncertainty, score convention	Researcher
Anomaly output	Versioned scores/events, explanations, severity, provenance	Security reviewer
Evaluation package	Protocol, baselines, metrics, uncertainty, injections, error analysis	Supervisor
Adaptive module	Safe buffer, shadow test, promotion, rollback, poisoning report	Security reviewer
Forecast/response module	Real actuals, validated metrics, advisory policy, audit trail	Supervisor
Edge package	Hardened image, SBOM, benchmark data, deployment guide	Deployment reviewer
Thesis evidence	Rebuildable LaTeX tables/figures tied to immutable manifests	Author/supervisor

15 Progress Summary

Phase	Outcome	Status
F0	Engineering and research foundations	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P0	Frozen reproducible research setup	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P1	Validated data-processing pipeline	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P2	Reproducible DM model training	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P3	Reliable pluggable likelihood engine	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P4	Calibrated online anomaly scoring	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P5	Controlled comparative evaluation	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P6	Guarded adaptive retraining	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P7	Forecasting and advisory response support	☐ Not started ☐ In progress ☐ Blocked ☐ Done
P8	Secure edge-deployment evidence	☐ Not started ☐ In progress ☐ Blocked ☐ Done